arXiv:2405.16287v1 [cs.LG] 25 May 2024

LoGAH: Predicting 774-Million-Parameter Transformers using Graph HyperNetworks with 1/100 Parameters

Xinyu Zhou EPFL
Boris Knyazev Samsung - SAIT AI Lab
Alexia Jolicoeur-Martineau Samsung - SAIT AI Lab
Jie Fu* HKUST

Abstract

A good initialization of deep learning models is essential since it can help them converge better and faster. However, pretraining large models is unaffordable for many researchers, which makes a desired prediction for initial parameters more necessary nowadays. Graph HyperNetworks (GHNs), one approach to predicting model parameters, have recently shown strong performance in initializing large vision models. Unfortunately, predicting parameters of very wide networks relies on copying small chunks of parameters multiple times and requires an extremely large number of parameters to support full prediction, which greatly hinders its adoption in practice. To address this limitation, we propose LoGAH (Low-rank GrAph Hypernetworks), a GHN with a low-rank parameter decoder that expands to significantly wider networks without requiring as excessive increase of parameters as in previous attempts. LoGAH allows us to predict the parameters of 774-million large neural networks in a memory-efficient manner. We show that vision and language models (i.e., ViT and GPT-2) initialized with LoGAH achieve better performance than those initialized randomly or using existing hypernetworks. Furthermore, we show promising transfer learning results w.r.t. training LoGAH on small datasets and using the predicted parameters to initialize for larger tasks. We provide the codes in `https://github.com/Blackzxy/LoGAH`.

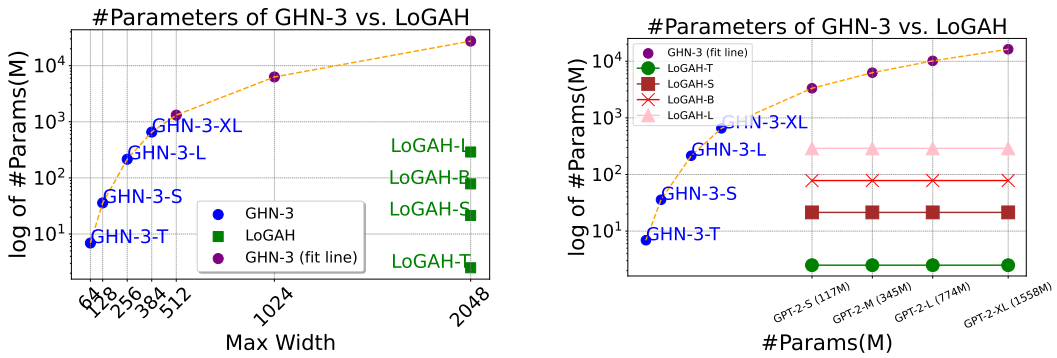
1 Introduction

In vision and language domains, pretraining a large model from scratch precedes solving downstream tasks [He et al., 2021, Devlin et al., 2019]. Recent models have been increasing in size dramatically, chasing state-of-the-art performance: from around 100M to $\geq$65B parameters for Generative Pre-trained Transformers (GPTs) [Radford et al., 2018, Touvron et al., 2023, AI@Meta, 2024] and from around 100M to $\geq$22B for Vision Transformers (ViTs) [Dosovitskiy et al., 2021, Dehghani et al., 2023]. Training such large models requires large computing resources. In addition, many retraining iterations are often required before the model is successfully trained, which is exacerbated in larger models since they are often more unstable to train and require more hardware and software tuning in addition to hyperparameter and architecture tuning, data curation, etc. Thus, pretraining large models has become very expensive even for big companies [Thompson et al., 2022, Zhai et al., 2022].

Currently, training vision and language tasks are generally done using similar network architectures and datasets; architectures are generally based on either Transformers [Vaswani et al., 2023] (for vision or language) or Convolutional Neural Networks (CNNs) [Fukushima et al., 1983] (for vision),

*Corresponding Author.

Preprint. Under review.



(a) Comparison of parameter counts between GHN-3 and LoGAH for supportable maximal widths in the predicted parameters (without their copying).

(b) Comparison of parameter counts between GHN-3 and LoGAH for supportable network sizes (without copying predicted parameters).

Figure 1: Comparison of parameter counts between GHN-3 and LoGAH. GHN-3 requires a larger hidden size to support wider networks, which increases the size of GHN-3 exponentially in Figure 1a. LoGAH can support much wider networks (up to 2048-dimension), and larger networks (GPT-2-Large in 1280-dimension with 774M parameters) even using LoGAH-Tiny.

while datasets are similar to either ImageNet (for vision) [Russakovsky et al., 2015] or The Pile (for language) [Gao et al., 2020]. Leveraging this prior knowledge of the architecture and dataset may reduce the pretraining cost. One approach to do so is Graph HyperNetworks (GHNs)[Zhang et al., 2018, Knyazev et al., 2021, 2023]; this approach allows one to predict initial parameters of these neural networks that perform well and converge faster. We describe the GHN approach below.

Using a set of neural network architectures $\{f^G\}$ as training data, GHN $H_{\mathcal{D}}$, parameterized by θ, is trained to predict the parameters of these neural networks ($\mathbf{w}_{\text{pred}} = H_{\mathcal{D}}(f^G, \theta)$) to minimize the loss function on the dataset $\mathcal{D}$. The predicted $\mathbf{w}_{\text{pred}}$ can serve as a stronger initialization compared to random-based initialization methods, thus greatly reducing the pretraining cost.

However, to predict parameters for very wide networks (often with a large number of parameters), previous GHNs [Knyazev et al., 2021, 2023] had to copy small chunks of parameters multiple times instead of fully predicting them due to the sheer amount of parameters required to predict all parameters, thus significantly limiting the performance of the resulting networks. Furthermore, to unlock the capability of predicting parameters of a larger size, GHNs need larger hidden sizes d, leading to an exponential increase in the number of parameters growing as $\mathcal{O}(d^3)$ (Figure 1a).

To overcome this limitation, we propose LoGAH, a GHN with a low-rank parameter decoder. This novel approach not only supports significantly wider networks but also does so without requiring an excessive number of parameters growing as $\mathcal{O}(d^2)$ instead of $\mathcal{O}(d^3)$ (Figure 1b). For instance, our smallest LoGAH-Tiny has only 2.5M parameters, yet it can predict parameters with up to 2048 channels, including GPT-2-Large with 774M parameters and potentially even larger networks.

In this work, we make the following contributions:

- We propose LoGAH, with an improved low-rank decoder, that is more scalable and can predict parameters of large networks without copying while having fewer trainable parameters and a lower training cost (Section 3).
- We create a new dataset of small ViT and GPT-2 architectures, allowing GHNs to be trained on Transformers for both vision and language domains (Section 4). LoGAH shows excellent generalized capability on larger models.
- We outperform GHN-3 as an initialization approach in multiple vision and language tasks by predicting more diverse and performant parameters (Section 5).

2 Preliminaries

2.1 Graph HyperNetworks

Graph HyperNetworks (GHNs) [Zhang et al., 2020, Knyazev et al., 2021] are widely used for neural networks' parameter prediction. The input fed to GHN $H_{\mathcal{D}}(\theta)$ is a computational graph f^G of a neural network f; GHN predicts its parameters $\mathbf{w}_{\text{pred}} = H_{\mathcal{D}}(f^G; \theta)$, where $\mathcal{D}$ is the training dataset. In our paper, f can be a ViT model [Dosovitskiy et al., 2021] (*resp.* GPT-2 [Radford et al., 2019]), and $\mathcal{D}$ can be the image classification task (*resp.* causal language modeling task).

In Knyazev et al. [2021] work, GHN $H_{\mathcal{D}}$ is trained by SGD over M training architectures $\{f_a^G\}_{a=1}^M$ and N training data samples $\{\mathbf{x}_j, y_j\}_{j=1}^N$ on the following optimization problem:

$$\arg\min_{\theta} \frac{1}{NM} \sum_{j=1}^{N} \sum_{a=1}^{M} \mathcal{L}(f_a(\mathbf{x}_j; H_{\mathcal{D}}(f_a^G; \theta)), y_j). \tag{1}$$

A meta-batch of m training architectures is sampled in the training stage where $H_{\mathcal{D}}$ predicts parameters. Meanwhile, a mini-batch of n training samples $\mathbf{x}$ is sampled and fed into the parameter-predicted m architectures to get $m \times n$ predictions. The cross-entropy loss $\mathcal{L}$ is computed for classification and language modeling tasks (next-token prediction). Afterward, the loss is back-propagated to update the parameters θ of $H_{\mathcal{D}}$ by gradient descent. In our work, we created ViTs-1K and GPTs-1K datasets of small training architectures for predicting parameters for larger ViT and GPT-2 models, respectively. We describe the details in Section 4.

The computational graph $f^G = (V, E)$ for input is a Directed Acyclic Graph (DAG), where V denotes the operations (e.g., pooling, self-attention, etc.), and E corresponds to the forward pass flow of inputs through f. The d-dimensional node features $\mathbf{H}^{(1)} \in \mathbb{R}^{|V| \times d}$ are obtained by an embedding layer (i-th node: $\mathbf{h}_i^{(1)} = \text{Embed}(\mathbf{h}_i^{(0)})$, where $\mathbf{h}_i^{(0)}$ is a one-hot vector representing for an operation) and fed as the input for GHN. After L Graphormer layers [Ying et al., 2021], the node features $\mathbf{H}^{(L)} \in \mathbb{R}^{|V| \times d}$ are fed to the decoder described below.

2.2 GHN Decoder

Knyazev et al. [2021, 2023] have the decoder based on a simple MLP predicting a tensor of shape $d \times d \times 16 \times 16$, where d is relatively small ($d = 384$ even in the largest GHN-3). The decoder takes the output node features of the last Graphormer layer to predict parameters $\mathbf{w}_{\text{pred}}$. This tensor is copied when the target weight has a larger d or sliced when the target is smaller. The parameter count of the decoder in [Knyazev et al., 2021, 2023][2] is:

$$\#\text{Param}_{\text{GHN-decoder}} = 4d^2 \times 16 \times 16 + 32d^2 + 8d^3 + d \times \text{num_class} \in \mathcal{O}(d^3). \tag{2}$$

3 Scalable Graph HyperNetworks: LoGAH

LoGAH model improves on the following aspects: (1) designing a novel low-rank decoder not only with fewer amounts of parameters, but also avoiding inefficient parameter repetitions on prediction, (2) supporting larger models (often wider) prediction without involving extremely larger amounts of parameters as in previous works, e.g. LoGAH-Tiny with only 2.5M parameters can support GPT-2-Large, while existing methods [Knyazev et al., 2023] would require at least $\sim 10^5$M parameters.

3.1 Low-Rank Decoder

In [Knyazev et al., 2023], the final output dimensionality of the decoder is $d \times d \times 16 \times 16$, where d can be 64 or 128. In most cases, 16×16 can be a waste since convolutional parameters are generally in 3×3 or 7×7. However, the bigger problem is that for large networks, the tensor needs to be repeated to fill all channels because d is small.

[2] Please refer to Appendix A and `https://github.com/SamsungSAILMontreal/ghn3/blob/main/ghn3/nn.py` for more details

Considering a convolutional weight W with size: $(C_{out} \times C_{in} \times h \times w)$, we can reshape it into a matrix W of $(C_{out} \cdot h) \times (C_{in} \cdot w)$ where h, w are much smaller than C_{out} and C_{in}. Inspired by [Hu et al., 2021], we can now introduce the low-rank decomposition:

$$W = AB \in \mathbb{R}^{(C_{out} \cdot h) \times (C_{in} \cdot w)}, \tag{3}$$

where $A \in \mathbb{R}^{(C_{out} \cdot h) \times r}, B \in \mathbb{R}^{r \times (C_{in} \cdot w)}$, r denotes the low-rank. In this way, we reduce the amounts of parameters from $C_{out} \cdot C_{in} \cdot h \cdot w$ to $r \cdot ((C_{out} \cdot h) + (C_{in} \cdot w))$.

Therefore, the whole process is as follows: after the first MLPs (multilayer perceptron) the input $\mathbf{H}^{(L)} \in \mathbb{R}^{|V| \times d}$ is transformed into $\tilde{W} \in \mathbb{R}^{|V| \times 2K \times r}$:

$$\tilde{W} = \text{MLP}(\mathbf{H}^{(L)}) \in \mathbb{R}^{|V| \times 2K \times r}, \tag{4}$$

where $K := \max(C_{out} \cdot h, C_{in} \cdot w)$ is called **max mask**, so that we can avoid repetition operations in GHN-3. Then we split $\tilde{W}$ into two matrices $A, B^T \in \mathbb{R}^{|V| \times K \times r}$ and only take the needed bits to construct $W = AB$ in Eqn. (3). The architecture of the MLPs is shown in Appendix B, which involves the low-rank transformation inside. In this way, the number of parameters in the decoder of LoGAH is:

$$\#\text{Param}_{\text{LoGAH-decoder}} = 4d^2 + 32d^2 + 8d \times 2r^2 + r \times K. \tag{5}$$

Theoretically, we can fix r as a much smaller constant hyperparameter than d, then Eqn. (5) would be in $\mathcal{O}(d^2)$, less than the complexity of original GHN's decoder $\mathcal{O}(d^3)$. In practice, considering a small rank r would hinder the model's performance, so we set it to $r \approx \frac{d}{2}$ as an increase of d. Under this setting, we compare the amounts of two decoder's parameters in detail as follows.

#Parameters Comparison. Without loss of generality, we assume $K = C_{out} \cdot h$, and in our following settings for low-rank r (details in Table 1)[3]:$r \approx \frac{d}{2}$. Then Eqn. (2) - Eqn. (5) we obtain:

$$\Delta\mathbf{P} = 4d^2 \times (16^2 - 1) + 8d \times (d^2 - 2r^2) + d \times \text{num_class} - r \times C_{out} \cdot h. \tag{6}$$

Since $r \approx d/2$, $16^2 - 1 \approx 16^2$, and in our experiments we set $K = \max(C_{out} \cdot h, C_{in} \cdot w) = 2048 \cdot 16$, we can just compare the first term and last term in Eqn . (6):

$$\begin{aligned} \Delta_1 &= 4d^2 \times (16^2 - 1) - r \times C_{out} \cdot h && (7) \\ &\approx 4d^2 \times 16^2 - d \times 1024 \cdot 16 && (8) \\ &= 16d \cdot (64d - 1024). && (9) \end{aligned}$$

Therefore, $\Delta_1 > 0$ since in our settings $d = 64, 128, 256$, etc, which means that LoGAH's decoder requires fewer parameters ($\Delta\mathbf{P} > 0$), even if we let r increase with d.

3.2 Predicting parameters in larger shapes with fewer parameters

Thanks to the low-rank mechanism, LoGAH can support predicting the parameter tensors with a larger shape but with fewer parameters. The parameters comparison between different versions of GHN-3 and LoGAH is shown in Figure 1. Since GHN-3 can only support the predicted parameters as the same width as the hidden size d, we fit the curve of GHN-3 and obtain the potential number of parameters needed to fully predict parameters with larger shapes. Compared to GHN-3, our LoGAH can support wider tensor shapes with much fewer parameters, which can support larger models (often wider) in practice (referring to Table 6 and Table 7).

4 ViTs-1K and GPTs-1K Datasets

For sampling training architectures in previous GHN-related works, Knyazev et al. [2021] built DeepNets-1M, a dataset of 1 million diverse computational graphs. However, for generating Transformer models such as ViT and GPT-2, DeepNets-1M is not optimal. Therefore we introduce

[3]Although in LoGAH-Large setting: $d = r = 256$, Eqn. (9) will obtain $16d \cdot (64d - 2048) > 0$ since d is very large. We also tried $d = 384, r = 256$, however, the training is unstable.

Table 1: Details of LoGAH variants and GHN-3 variants. All LoGAH variants are set with $K = 2048 \cdot 16$. We estimate the train time of each model based on meta-batch $m = 1$ and the CIFAR-100 dataset for 300 epochs.

Model	r	L	d	H	Max Width	P	Train Time
LoGAH-Tiny	32	3	64	8	2048	2.5M	7.05h
LoGAH-Small	90	5	128	16	2048	21.4M	7.25h
LoGAH-Base	128	5	256	16	2048	78.2M	10.30h
LoGAH-Large	256	12	256	16	2048	289.4M	21.0h
GHN-3-Tiny	-	3	64	8	64	6.9M	7.20h
GHN-3-Small	-	5	128	16	128	35.8M	7.75h
GHN-3-Large	-	12	256	16	256	214.7M	12.40h
GHN-3-XLarge	-	24	384	16	384	654.4M	24.0h

ViTs-1K and GPTs-1K: these new datasets contains 1K different ViT-style and GPT-2-style computational graphs respectively, particularly for training GHNs to predict ViT and GPT-2's parameters.

ViTs-1K. We produce diverse ViT models by varying the number of layers L, heads H and hidden size D. Since ViT models have different scale versions (as illustrated in Table 6 of Appendix C), we also need to ensure that our training architectures will be diverse enough and uniformly distributed in terms of parameter count. Therefore, when generating these architectures, for deeper networks (with more layers) we control them to be narrower (with a smaller hidden size) and vice versa. Figure 8a shows the distribution of the amounts of parameters in ViTs-1K, which is almost uniformly distributed and the maximum parameters of these architectures are restricted to 10M (only around of half of ViT-Small's parameters). The details of ViTs-1K dataset's generation can be found in Appendix E.

GPTs-1K. We follow the same above idea to get different GPT-2 models, by varying the number of layers L, heads H and hidden size D, to build GPTs-1K. The parameter count distribution is shown in Figure 8b, and the maximum parameter count is within 30M, which is much less than GPT2-Small with 110M parameters. The GPT-2 variants details are presented in Table 7 in Appendix C and the ones for the GPTs-1K dataset's generation can be found in Appendix F.

Importantly, these datasets are smaller than Large Language Models (LLMs) and similarly large vision models; this is by design. The purpose is to reduce the computation required for learning to predict parameters while giving a continuous range of scale (from tiny to large) so that LoGAH can generalize to large models after training.

5 Experiments

We evaluate if networks (i.e. ViT and GPT-2) initialized with the parameters $\mathbf{w}_{\text{pred}}$ predicted by LoGAH can perform better than those by GHN-3 and random initialization after fine-tuning.

LoGAH Variants. We provide four different scales of LoGAH from Tiny to Large, by gradually increasing the number of layers L, hidden size d, heads H, as well as the low-rank r. We also compare the number of parameters and estimate the training time difference between LoGAH with GHN-3, shown in Table 1. We highlight that GHN-3 and LoGAH are trained only once on each dataset, so that the same model can predict parameters for many architectures making the training cost of GHN-3 and LoGAH amortized.

GHN Training Setup. The GHN models, including GHN-3 and our LoGAH, are trained for 300 epochs on ViTs-1K and GPTs-1K datasets. For ViT, we conduct experiments on the following datasets: CIFAR-10, CIFAR-100 [Krizhevsky et al., 2009] (with batch size $b = 64$) and ILSVRC-2012 ImageNet [Russakovsky et al., 2015] (with batch size $b = 128$). We train the models using automatic mixed precision in PyTorch with a cosine annealing learning rate schedule starting at $lr = 3e-4$, weight decay $\lambda = 1e-2$, and predicted parameter regularization $\gamma = 3e-5$ [Knyazev et al., 2023]. For GPT-2 experiments we use the WikiText dataset [Merity et al., 2016], and use

Table 2: CIFAR-10, CIFAR-100 and ImageNet top-1 accuracy (%) on ViT-Small and ViT-Base in different initialization settings. ViT models on CIFAR datasets are fine-tuned for 100 epochs in each initialization setting, while for ImageNet dataset, ViTs are fine-tuned for 30 epochs due to the time cost consideration. All GHN models can be trained on a single NVIDIA 4090 GPU when meta-batch $m = 1$ with /m1 suffix. On ImageNet, we only train GHN-3-Tiny.

Initialization	CIFAR-10		CIFAR-100		ImageNet	
	ViT-Small	ViT-Base	ViT-Small	ViT-Base	ViT-Small	ViT-Base
RANDINIT	83.93	84.74	58.73	57.00	62.15	63.61
ORTHINIT	80.11	84.24	58.97	57.48	63.15	63.86
GHN-3-T/m1	81.29	78.18	56.33	56.70	38.79	31.89
GHN-3-S/m1	83.61	82.57	57.37	59.93	-	-
GHN-3-L/m1	84.69	82.84	57.48	56.57	-	-
LoGAH-T/m1	82.87	82.87	60.05	56.77	62.16	61.68
LoGAH-S/m1	**86.09**	**86.35**	61.05	62.38	62.49	63.29
LoGAH-B/m1	85.16	85.39	60.58	60.02	59.00	61.22
LoGAH-L/m1	83.22	84.40	**62.19**	**63.47**	63.00	63.70

$lr = 1e-4$ with batch size $b = 6$, while keeping the other hyperparameters as before. All GHN models, including GHN-3 and LOGAH, are trained separately on each task dataset.

5.1 ViT Experiments.

5.1.1 Overall Comparision on CIFAR-10, CIFAR-100 and ImageNet

We test ViT-small and ViT-base on CIFAR-10, CIFAR-100 [Krizhevsky et al., 2009] and ILSVRC-2012 ImageNet [Russakovsky et al., 2015] shown in Table 2 with different initialization methods: (1) random initialization (RANDINIT) implemented by default in PyTorch, (2) orthogonal initialization (ORTHINIT) [Saxe et al., 2014], (3) parameters predicted by GHN-3, and (4) parameters predicted by LOGAH. From Table 2, we observe that LOGAH generally outperforms RANDINIT, ORTHINIT and GHN-3.

The ViT models are fine-tuned for 100 epochs on CIFAR, and 30 epochs on ImageNet datasets. For ViT-Small, we set the learning rate range as $\{0.1, 0.2, 0.3, 0.4, 0.5\}$. For ViT-Base and ViT-Large, we set it as $\{0.03, 0.04, 0.05, 0.06, 0.1\}$. All ViT models are fine-tuned using SGD with the same setting as Knyazev et al. [2023], but with weight decay $1e-2$. We report the best validation accuracy among all learning rates.

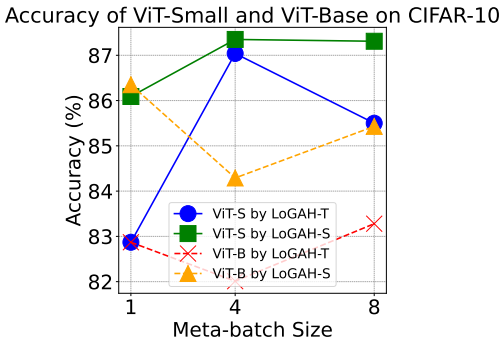


Figure 2: CIFAR-10 top-1 accuracy (%) on ViT-Small and ViT-Base where LOGAH is trained with different meta-batch size m.

CIFAR-10. LOGAH-SMALL presents the best performance (i.e. 2.16% improvement than RANDINIT, and 1.4% improvement than GHN-3-L in ViT-small) with only 21.41M parameters, compared to GHN-3-L with 214.7M parameters, 10x larger than LOGAH-SMALL, which demonstrates the effectiveness of our low-rank decoder. For ViT-base models, GHN-3-L has the closest performance to our smallest version of our models: LOGAH-TINY.

CIFAR-100. LOGAH-LARGE outperforms other models with a big improvement, especially compared to RANDINIT (3.46% on ViT-Small and 6.47% on ViT-Base), while GHN-3's performances are even worse than the random initializations on ViT-Small.

ImageNet. Considering the time cost for training GHNs on ImageNet, for GHN-3, we only train GHN-3-T/m1 in this dataset for comparison. The random initialization methods, including RANDINIT and ORTHINIT, show comparable performances with LOGAH. In this dataset, we do not observe a big improvement as in CIFAR-10 and CIFAR-100. However, LOGAH-T performs much better than GHN-3-T (62.16 vs. 38.79 for ViT-Small).

5.1.2 Effect of meta-batch size m on LOGAH

In this section, we study the effect of the meta-batch size m of LOGAH-TINY and LOGAH-SMALL on ViT-Small, ViT-Base and ViT-Large (shown in Figure 2 for CIFAR-10 and Table 3 for CIFAR-100). We increase the value of m from 1 to $4, 8$ and train LOGAH respectively.

CIFAR-10. In Figure 2, we can notice that increasing the meta-batch size m can significantly improve the ViT-Small's performance (i.e. from 82.87 to 87.04 by LOGAH-TINY, and from 86.09 to 87.35 by LOGAH-SMALL when setting $m = 4$), which is even better than larger LOGAH models trained with $m = 1$. However, a reverse pattern is observed in ViT-Base, increasing m to 4 will worsen the performance on CIFAR-10.

CIFAR-100. For the CIFAR-100 dataset, we extend to the larger model: ViT-Large. Enlarging the meta-batch size properly can significantly improve the small LOGAH's performance and help it achieve similar results as the larger one (e.g. LOGAH-T/M4 vs. LOGAH-S/M1). Another interesting finding is that when setting the meta-batch size $m = 8$, both LOGAH's performance drops dramatically. One potential reason might lie in the smaller model sizes in VITS-1K, and larger meta-batch may cause the overfitting problem.

Table 3: CIFAR-100 top-1 accuracy (%) on ViT-Base and ViT-Large where $m = 4, 8$ for training LOGAH.

Initialization	ViT-Base	ViT-Large
RANDINIT	57.00	55.62
LoGAH-T/m1	56.77	55.07
LoGAH-T/m4	60.61	59.88
LoGAH-T/m8	51.12	50.39
LoGAH-S/m1	**62.38**	**59.90**
LoGAH-S/m4	55.00	54.62
LoGAH-S/m8	51.58	51.61

5.2 GPT-2 Experiments

Considering the time and other resource costs[4], we only apply two smallest LOGAH, LOGAH-T/M2 and LOGAH-S/M2, in the GPT-2 experiments. This section investigates the Causal Language Modeling (CLM) task. We test the model's performance on the WikiText dataset[Merity et al., 2016]. In detail, we choose GPT-2-Small and GPT-2-Medium for the `wikitext-2-raw-v1` dataset. For the `wikitext-103-raw-v1` dataset, we select GPT-2-Medium and GPT-2-Large. All models are trained with the randomly-initialized parameters (i.e. RANDINIT), which is implemented by default in HuggingFace [Wolf et al., 2019].

GPT-2 Training Setup. All the GPT-2 models are fine-tuned for 100 epochs on each dataset. We use DeepSpeed [Rajbhandari et al., 2020] for improved training efficiency with GPT-2-Medium and GPT-2-Large on WikiText-103. With $6\times$ NVIDIA 4090 GPUs, we train the models by AdamW with learning rate as $3e-6$, weight decay as $1e-2$, batch size as 4 for GPT-2-Medium and as 2 for GPT-2-Large.

Results. Table 4 shows the results. The performance is improved more significantly on larger GPT-2 models and larger datasets (e.g. training GPT-2-Large on the WikiText-103 dataset), which demonstrates that even our smallest model (2.5M) can predict much better parameters than a random initialization for a large model (774M).

5.3 Qualitative Analysis

We analyze the diversity of predicted parameters following experiments in Knyazev et al. [2023, 2021]. In detail, we predict parameters for ViT and GPT-2, and collect one or two frequently occurring tensor shapes in each model. Then we compute the absolute cosine distance between all pairs of parameter

[4]For example, we did not train on the OpenWebText dataset [Gokaslan et al., 2019], since it is too large.

Table 4: Perplexity score of the GPT-2 experiments.

Initialization	WikiText-2		WikiText-103	
	GPT-2-Small	GPT-2-Medium	GPT-2-Medium	GPT-2-Large
RANDINIT	250	350	22.32	32.41
LoGAH-T/m2	**227**	**219**	**18.79**	27.18
LoGAH-S/m2	238	284	19.89	**24.08**

Table 5: Diversity of the parameters predicted by GHN-3, LOGAH vs. Pretrained ($*$: or trained by SGD from RANDINIT if pretrained parameters are unavailable) measured on the ViT and GPT-2. Pretrain*: ViT-Small is trained by SGD from RANDINIT for 100 epochs on CIFAR-100; for ViT-Base and ViT-Large we use the parameters pretrained on ImageNet; for GPT-2-Medium, we also load the available pretrained parameters in HuggingFace [Wolf et al., 2019].

Method	**Parameter Tensor Shape**					
	CIFAR-100 ViT-S (1536,384)	ImageNet ViT-B (768,768)	ImageNet ViT-L (1024,1024)	(3072,1024)	WikiText-103 GPT-2-M (1024,1024)	(1024,4096)
Pretrain*	0.647	0.747	0.878	0.842	0.839	0.894
GHN-3-T	0.191	0.513	0.402	0.420	-	-
GHN-3-S	0.290	-	-	-	-	-
LOGAH-T	0.284	**0.563**	**0.410**	0.385	**0.420**	0.393
LOGAH-S	**0.442**	0.413	0.232	0.329	0.400	**0.496**

tensors of the same shape and average it (Table 5). LOGAH predicts more diverse parameters than GHN-3 in general on ViT models, especially in ViT-Small on CIFAR-100, which is also consistent with the better performance in Table 2. Another interesting finding is that LOGAH-TINY is better at predicting more diverse square parameters (e.g. (768,768) and (1024,1024)) than LOGAH-SMALL.

5.4 Transfer Learning Experiments

In this section, we explore the setting when LOGAH is trained on one dataset, but is used to produce a parameter initialization for another (potentially more difficult) dataset. For the ViT experiments, we conduct the transfer learning experiments from CIFAR-10 to CIFAR-100, and from CIFAR-100 to ImageNet. For the GPT-2 experiments, we consider experiments from WikiText-2 to WikiText-103.

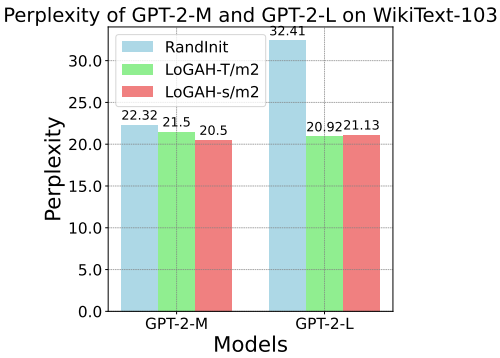


Figure 3: GPT-2 transfer learning experiments. LOGAH are trained on WikiText-2 and GPT-2 models are fine-tuned on WikiText-103 based on LOGAH's predicted parameters.

ViT Experiments. In detail, we re-initialize the classification layer [Knyazev et al., 2023] using a Kaiming normal distribution [He et al., 2015] with 100 and 1,000 outputs, for transferring to CIFAR-100 and ImageNet respectively, then we fine-tune the entire network. The results are shown in Figure 4. On CIFAR-100, ViTs initialized by the LOGAH-S, LOGAH-B outperform random initialization. It is still a demanding task for LOGAH to work on the ImageNet dataset, and LOGAH-T is generally better than larger ones, which suggests that parameters that work well on CIFAR-100 may not be optimal for ImageNet. However, compared to GHN-3-T in Table 2, it shows superior performances in both ViT-Small and ViT-Base.

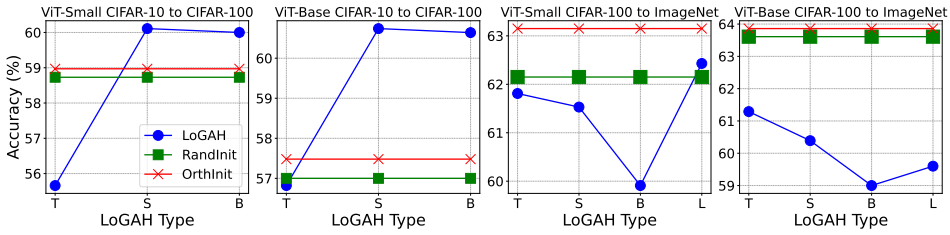


Figure 4: ViT transfer learning experiments. We use LoGAH trained on CIFAR-10 (*resp.* CIFAR-100) to predict ViT's parameters, then ViT is trained on CIFAR-100 (*resp.* ImageNet). T, S, B and L denotes TINY, SMALL, BASE and LARGE versions of LoGAH respectively.

.

GPT-2 Experiments. We keep the same setting as in Section 5.2, and fine-tune GPT-2-Medium and GPT-2-Large on WikiText-103 loaded with LoGAH predicted parameters, which are trained on the WikiText-2 dataset. According to Figure 3, LoGAH's predicted parameters are also a good initialization for fine-tuning GPT-2 models on WikiText-103, especially for GPT-2-Large.

According to the above results, the parameters predicted by LoGAH present a desired transfer learning ability from one easier to another harder task. This improvements are more significant if the data distribution is close (e.g. from CIFAR-10 to CIFAR-100). Moreover, this property can help reduce the training time for LoGAH to predict good parameters, where we do not need to train LoGAH on a large-scale dataset.

6 Related Work

Large Models Pretraining. The large-scale pretrained models first appeared in the NLP field [Yin et al., 2022, Guo et al., 2022]. The improvement and success are mainly attributed to self-supervised learning and Transformer [Vaswani et al., 2023]. More and more large language models are developed based on it, extending to larger sizes for better performance under pretraining with massive data [Devlin et al., 2019, Brown et al., 2020, Touvron et al., 2023]. Inspired by the advancement of Transformer, many Transformer-based vision models are also proposed, and some pretraining methods have been explored [Dosovitskiy et al., 2021, Carion et al., 2020, He et al., 2021, Chen et al., 2020]. Our work focuses on predicting parameters for two Transformer-based models (ViT and GPT-2) to reduce pretraining costs.

Parameter Prediction. Hypernetworks [Ha et al., 2016] are often leveraged for predicting model's parameter. Many research works have extended the hypernetwork's capability to generalize on unseen architectures [Zhang et al., 2018, Nirkin et al., 2021, Knyazev et al., 2021], datasets [Requeima et al., 2020, Lin et al., 2021, Zhmoginov et al., 2022, Kirsch et al., 2024], or to generate interpretable networks [Liao et al., 2023]. Our paper is also based on Graph HyperNetworks (GHNs), but overcomes the extreme increase of parameters needed in previous GHNs. LoGAH can support larger models with just 1% parameters, showing a better ability to predict parameters for larger networks.

Initialization and Learning to Grow Models. Several methods have improved on random initialization by learning from data [Dauphin and Schoenholz, 2019, Yang et al., 2022]. However, GHN-3 [Knyazev et al., 2023] showed better performance making it a favourable approach to build on. Other methods learn to initialize a bigger model from a smaller pretrained model [Evci et al., 2022, Wang et al., 2023]. These methods reduce training time, however, a smaller pretrained model of exactly the same architecture as the target model is not always available, which limits the approach.

7 Limitations

Although our model LoGAH shows outstanding performances compared to GHN-3 and other random initialization methods across the extensive experiments, there are still limitations. Most importantly, due to the consideration of time and resource costs, we conduct GPT-2 experiments only on the WikiText dataset and only with our two smallest models. Furthermore, in order to predict parameters for drastically novel architectures (e.g. [Gu and Dao, 2023]), the GHN might be needed to be trained to avoid a big distribution shift. In future work, it would be intriguing to show LoGAH's ability on modern LLMs [Touvron et al., 2023].

8 Conclusion

In this work, we propose LoGAH, a low-rank Graph HyperNetwork (GHN) that addresses two issues of previous GHN-3. First, the low-rank decoder avoids copying small chunks of parameters multiple times when predicting a large shape parameter. Second, contrary to GHN-3, it does not require an exceptional number of trainable parameters to support wider or larger models, and our smallest LoGAH is only about 2.5M. We conduct extensive experiments on two representative transformer-based models (ViT in vision and GPT-2 in language) to show superior efficacy compared to GHN-3 and random initialization methods. Furthermore, the generalization ability of LoGAH from a simple to another more difficult dataset is also verified in the transfer learning experiments.

References

Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners, 2021. 1, 9

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In Jill Burstein, Christy Doran, and Thamar Solorio, editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1423. URL https://aclanthology.org/N19-1423. 1, 9

Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. 2018. 1

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023. 1, 9

AI@Meta. Llama 3 model card. 2024. URL https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md. 1

Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021. 1, 3, 9

Mostafa Dehghani, Josip Djolonga, Basil Mustafa, Piotr Padlewski, Jonathan Heek, Justin Gilmer, Andreas Peter Steiner, Mathilde Caron, Robert Geirhos, Ibrahim Alabdulmohsin, et al. Scaling vision transformers to 22 billion parameters. In *International Conference on Machine Learning*, pages 7480–7512. PMLR, 2023. 1

Neil C. Thompson, Kristjan Greenewald, Keeheon Lee, and Gabriel F. Manso. The computational limits of deep learning, 2022. 1

Xiaohua Zhai, Alexander Kolesnikov, Neil Houlsby, and Lucas Beyer. Scaling vision transformers, 2022. 1

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023. 1, 9

Kunihiko Fukushima, Sei Miyake, and Takayuki Ito. Neocognitron: A neural network model for a mechanism of visual pattern recognition. *IEEE transactions on systems, man, and cybernetics*, (5): 826–834, 1983. 1

Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, et al. Imagenet large scale visual recognition challenge. *International journal of computer vision*, 115:211–252, 2015. 2, 5, 6

Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, et al. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020. 2

Chris Zhang, Mengye Ren, and Raquel Urtasun. Graph hypernetworks for neural architecture search. *arXiv preprint arXiv:1810.05749*, 2018. 2, 9

Boris Knyazev, Michal Drozdzal, Graham W. Taylor, and Adriana Romero-Soriano. Parameter prediction for unseen deep architectures, 2021. 2, 3, 4, 7, 9

Boris Knyazev, Doha Hwang, and Simon Lacoste-Julien. Can we scale transformers to predict parameters of diverse imagenet models?, 2023. 2, 3, 5, 6, 7, 8, 9

Chris Zhang, Mengye Ren, and Raquel Urtasun. Graph hypernetworks for neural architecture search, 2020. 3

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019. 3

Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. Do transformers really perform bad for graph representation?, 2021. 3

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021. 4

Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009. 5, 6

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016. 5, 7

Andrew M. Saxe, James L. McClelland, and Surya Ganguli. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks, 2014. 6

Aaron Gokaslan, Vanya Cohen, Ellie Pavlick, and Stefanie Tellex. Openwebtext corpus. `http://Skylion007.github.io/OpenWebTextCorpus`, 2019. 7

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. Huggingface's transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771*, 2019. 7, 8

Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models, 2020. 7

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification, 2015. 8

Da Yin, Li Dong, Hao Cheng, Xiaodong Liu, Kai-Wei Chang, Furu Wei, and Jianfeng Gao. A survey of knowledge-intensive nlp with pre-trained language models, 2022. 9

Shangwei Guo, Chunlong Xie, Jiwei Li, Lingjuan Lyu, and Tianwei Zhang. Threats to pre-trained language models: Survey and taxonomy, 2022. 9

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020. 9

Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers, 2020. 9

Mark Chen, Alec Radford, Rewon Child, Jeffrey Wu, Heewoo Jun, David Luan, and Ilya Sutskever. Generative pretraining from pixels. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 1691–1703. PMLR, 13–18 Jul 2020. URL `https://proceedings.mlr.press/v119/chen20s.html`. 9

David Ha, Andrew Dai, and Quoc V. Le. Hypernetworks, 2016. 9

Yuval Nirkin, Lior Wolf, and Tal Hassner. Hyperseg: Patch-wise hypernetwork for real-time semantic segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4061–4070, June 2021. 9

James Requeima, Jonathan Gordon, John Bronskill, Sebastian Nowozin, and Richard E. Turner. Fast and flexible multi-task classification using conditional neural adaptive processes, 2020. 9

Xixun Lin, Jia Wu, Chuan Zhou, Shirui Pan, Yanan Cao, and Bin Wang. Task-adaptive neural process for user cold-start recommendation. In *Proceedings of the Web Conference 2021*, WWW '21, page 1306–1316, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450383127. doi: 10.1145/3442381.3449908. URL `https://doi.org/10.1145/3442381.3449908`. 9

Andrey Zhmoginov, Mark Sandler, and Max Vladymyrov. Hypertransformer: Model generation for supervised and semi-supervised few-shot learning, 2022. 9

Louis Kirsch, James Harrison, Jascha Sohl-Dickstein, and Luke Metz. General-purpose in-context learning by meta-learning transformers, 2024. 9

Isaac Liao, Ziming Liu, and Max Tegmark. Generating interpretable networks using hypernetworks. *arXiv preprint arXiv:2312.03051*, 2023. 9

Yann N Dauphin and Samuel Schoenholz. Metainit: Initializing learning by learning to initialize. *Advances in Neural Information Processing Systems*, 32, 2019. 9

Yibo Yang, Hong Wang, Haobo Yuan, and Zhouchen Lin. Towards theoretically inspired neural initialization optimization. *Advances in Neural Information Processing Systems*, 35:18983–18995, 2022. 9

Utku Evci, Bart van Merrienboer, Thomas Unterthiner, Max Vladymyrov, and Fabian Pedregosa. Gradmax: Growing neural networks using gradient information. *arXiv preprint arXiv:2201.05125*, 2022. 9

Peihao Wang, Rameswar Panda, Lucas Torroba Hennigen, Philip Greengard, Leonid Karlinsky, Rogerio Feris, David Daniel Cox, Zhangyang Wang, and Yoon Kim. Learning to grow pretrained models for efficient transformer training. *arXiv preprint arXiv:2303.00980*, 2023. 9

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023. 9

Kunihiko Fukushima. Cognitron: A self-organizing multilayered neural network. *Biological cybernetics*, 20(3-4):121–136, 1975. 13

A Details of the amounts of parameters of decoders in GHN-3

The theory amount of parameters of decoders in GHN-3 is shown below:

$$4 \times \text{in_feature} \times d \times h \times w + \text{MLP_d}_1 \times \text{MLP_d}_2 + \text{MLP_d}_2 \times d^2 + d \times \text{num_class} \tag{10}$$

where in_feature is the input feature's dimension of the decoder (set as d in GHN-3), and $\text{MLP_d}_1, \text{MLP_d}_2$ denote the dimension of 1_{st} and 2_{nd} layers of MLP (set as $4d$ and $8d$ in experiments respectively), h, w are the last two dimensions of the predicted tensor's shape (set as 16) and num_class is the number of classes of the dataset. Thereby, we can simplify Equation (10) to (2).

B Details of MLPs in the decoder of LoGAH

The MLPs has 4 layers and the activation function $\sigma(\cdot)$ is ReLU Fukushima [1975]:

$$\mathbf{x} = M_3\Big(\sigma\Big(M_2\Big(\sigma\big(M_1(\mathbf{H})\big)\Big)\Big)\Big) \tag{11}$$

$$\mathbf{x} = \text{reshape}(\mathbf{x}) \in \mathbb{R}^{|V| \times 2r \times r} \tag{12}$$

$$\mathbf{x} = \text{reshape}\big(M_4(\sigma(\mathbf{x}))\big) \in \mathbb{R}^{|V| \times 2K \times r} \tag{13}$$

where $M_i, i \in \{1, 2, 3, 4\}$ are learnable matrices:

$$M_1 \in \mathbb{R}^{d \times 4d}, M_2 \in \mathbb{R}^{4d \times 8d}$$
$$M_3 \in \mathbb{R}^{8d \times 2r^2}, M_4 \in \mathbb{R}^{r \times K}$$

We also provide the code implementation of it as shown in Figure 5.

C Details of variants of ViT and GPT-2

We provide the details of ViT and GPT-2 in different sizes. L, D, H, P denotes the numbers of layers, heads, hidden size and parameters, respectively.

Model	*L*	*D*	MLP size	*H*	P
ViT-S	12	384	1536	6	22M
ViT-B	12	768	3072	12	86M
ViT-L	24	1024	4096	16	307M

Table 6: Details of ViT variants

Model	*L*	*D*	*H*	P
GPT-2-S	12	768	12	110M
GPT-2-M	24	1024	16	345M
GPT-2-L	36	1280	20	774M

Table 7: Details of GPT-2 variants

D Distribution of VITS-1K and GPTS-1K datasets

The distributions of VITS-1K and GPTS-1K datasets are shown in Figure 8.

E Details of generating VITS-1K dataset

As mentioned above, we change the values in layers L, heads H, and hidden size D of ViT, as well as restricting these models size. The details are shown in Figure 6.

F Details of generating GPTS-1K dataset

We also change the values in layers L, heads H and hidden size D of GPT-2, and the details are shown in Figure 7.

```python
class ConvDecoder3LoRA(nn.Module):
    def __init__(self,
                 in_features,
                 ck=32,
                 r=32,
                 hid=(64,),
                 num_classes=None):
        super(ConvDecoder3LoRA, self).__init__()

        assert len(hid) > 0, hid
        self.r = r
        self.ck = ck
        self.num_classes = num_classes
        self.mlp = MLP(in_features=in_features,
                       hid=(*hid, r*2*r),
                       activation='relu',
                       last_activation=None)

        self.l2 = nn.Linear(int(r), ck)
        self.relu = nn.ReLU(inplace=True)

        self.seq = nn.Sequential(
            self.relu,
            self.l2
        )


    def forward(self, x, max_shape=(1,1,1,1), class_pred=False, n_dim = 4):
        if class_pred:
            n_dim = 2
        x = self.mlp(x).view(-1, 2*self.r, self.r) # [b, 2*r, r]
        x = self.seq(x).view(-1, 2*self.ck, self.r) # [b, 2*ck, r]
        A, B_t = torch.split(x, self.ck, dim=1) # A=[b, ck, r] and B=[b, ck, r]
        B = B_t.transpose(1,2) # A=[b, ck, r] and B=[b, r, ck]
        # fix shape of A and B before matmul through indexing
        c_out, c_in, k_out, k_in = max_shape
        A = A[:, :(c_out*k_out), :] # [b, c_out*k_out, r]
        B = B[:, :, :(c_in*k_in)] # [b, r, c_in*k_in]
        W = torch.bmm(A, B) # [b, c_out*k_out, c_in*k_in]
        if n_dim == 1: # We want [c_out]
            assert c_in == 1 and k_out == 1 and k_in == 1
            W = W.reshape(-1, c_out)
        elif n_dim == 2: # we already have a 2D matrix
            pass
        elif n_dim == 4:
            W = W.reshape(-1, c_out, k_out, c_in, k_in).transpose(2, 3) # [b, c_out,
                c_in, k_out, k_in]
        else:
            raise NotImplementedError("n_dim must be 1 or 2 or 3")
        #print(W.shape)
        return W
```

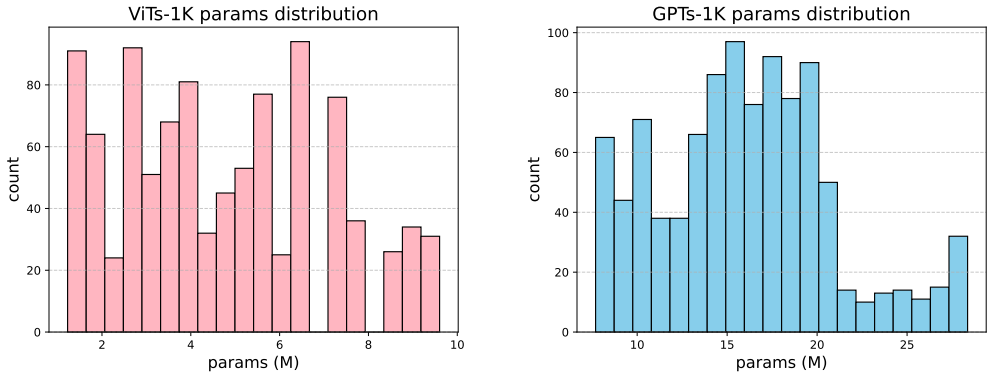
Figure 5: Code for Low-rank decoder in LoGAH.

```
layers = np.random.randint(3, 10)
if layers > 5:
    dim_min = 128
    dim_max = 256
elif layers > 3:
    dim_min = 256
    dim_max = 384
else:
    dim_min = 384
    dim_max = 512

hidden_dim = np.random.choice(np.arange(dim_min, dim_max+1, 32))
mlp_dim = hidden_dim * 4

if hidden_dim % 12 == 0:
    heads = np.random.choice([3, 6, 12])
elif hidden_dim % 6 == 0:
    heads = np.random.choice([3, 6])
elif hidden_dim % 3 == 0:
    heads = 3
else:
    heads = np.random.choice([4, 8])



net = _vision_transformer(
    patch_size = 2,
    num_layers = layers,
    num_heads = heads,
    hidden_dim = hidden_dim,
    mlp_dim = mlp_dim,
    num_classes = 100,
    image_size = 32,
    weights = None,
    progress = False,
)
```

Figure 6: Code for generating ViT-style models used for ViTs-1K dataset.

```
n_layer = np.random.randint(3, 10)

if n_layer > 5:
    dim_min = 72
    dim_max = 176
elif n_layer > 3:
    dim_min = 128
    dim_max = 176
else:
    dim_min = 176
    dim_max = 256

n_embd = np.random.choice(np.arange(dim_min, dim_max+1, 8))

if n_embd % 8 == 0:
    n_head = 8
elif n_embd % 6 == 0:
    n_head = 6
elif n_embd % 4 == 0:
    n_head = 4

config = GPT2Config(
    bos_token_id=tokenizer.bos_token_id,
    eos_token_id=tokenizer.eos_token_id,
    n_embd=int(n_embd),
    n_layer=int(n_layer),
    n_head=int(n_head),
    tie_word_embeddings=False,
)
model = GPT2LMHeadModel(config)
```

Figure 7: Code for generating GPT-2-style models used for GPTs-1K dataset.



(a) The parameters distribution in ViTs-1K.

(b) The parameters distribution in GPTs-1K.

Figure 8: The parameters distribution of ViTs-1K and GPTs-1K datasets